Introducción a los sistemas operativos

Clase 1

Los **sistemas operativos** son programas especiales que actúan como intermediario entre el usuario/aplicaciones y el hardware. Requiere procesador y memoria para ejecutarse.

Además, el SO proporciona un **conjunto de servicios** a los usuarios del sistema: permite crear y eliminar procesos, asignar memora, lee y escribir archivos, etc. Por lo tanto, el SO actúa como intermediario entre los programas de aplicación y el hardware.

Hay dos perspectivas

1. **De arriba hacia abajo (perspectiva del usuario)**

   El SO actúa como una **abstracción** respecto a la arquitectura del hardware. La arquitectura comprende el conjunto de instrucciones del procesador, la organización de la memoria, los dispositivos de E/S y la estructura del bus del sistema.

   El SO oculta toda esta complejidad del hardware y presenta a los programas de aplicación (que actúan como clientes del SO) abstracciones mucho más simples de manejar. El SO proporciona “comodidad” o “amigabilidad” al ocultar la complejidad y permitir que las aplicaciones se ejecuten de manera independiente de los detalles específicos del hardware.

2. **De abajo hacia arriba**

   En esta perspectiva, el SO se concibe como un **administración de recursos.** El hardware de una computadora posee recursos finitos: procesador, memoria principal, dispositivos de almacenamiento secundario y dispositivos de E/S. Cuando múltiples procesos compiten por estos recursos limitados, el SO debe arbitrar y distribuirlos eficiente y equitativamente.

   El SO administra estos recursos a varios procesos, asignando a cada uno la cantidad de recursos necesaria. Para esto, implementa un mecanismo llamado **multiplexación en tiempo:** divide el tiempo disponible del procesador en pequeños intervalos y le asigna a cada proceso se “turno” para ejecutarse. De manera similar, realiza **multiplexación en espacio** con la memoria, dividiendo el espacio disponible entre los procesos para que cada uno tenga su propia región de memoria aislada.

Objetivos de los SO

- Comodidad: hacer más fácil el uso del hardware.
- Eficiencia: hacer un uso más eficiente de los recursos del sistema.
- Evolución: permitir la introducción de nuevas funciones al sistema sin interferir con funciones anteriores.

Componentes de un SO

1. **Kernel (núcleo):** el kernel es el **corazón del SO** y la parte más crítica. Es el responsable de las funciones fundamentales: administración de procesos, gestión de memoria, manejo de interrupciones, control de dispositivos de E/S, sincronización entre procesos y protección de recursos.
2. **Shell (intérprete de comandos):** es la **interfaz entre el usuario y el kernel.** Actúa como un traductor que toma las órdenes del usuario y las convierte en llamadas al kernel. Hay 3 formas principales de interactuar con el SO
   - **GUI (interfaz gráfica de usuarios):** el shell gráfico presenta ventanas, iconos, menús y elementos visuales.
   - **CUI (interfaz de caracteres):** presenta formularios basados en texto donde el usuario ingresa datos.

- **CLI (interfaz de línea de comandos):** el usuario escribe comandos textuales que el shell interpreta y ejecuta como bash o powershell.

El shell **no es parte del kernel,** sino una capa intermedia que facilita la comunicación.

3. **Herramientas y utilidades:** más allá del kernel y la shell, el SO proporciona un **conjunto de herramientas y programas** que facilitan el desarrollo y uso del sistema. Estas incluyen editores de texto, compiladores, librerías estándar, depuradores, administradores de archivos, etc. No son esenciales para que el SO funcione si no hacen que sea práctico y productivo para los usuarios.

Servicios de un SO

El SO proporciona una serie de servicios fundamentales que permiten que los programas se ejecuten de manera ordenada, eficiente y sin conflictos. El principal servicio es la **administración y planificación del procesador,** que comprende varios aspectos interconectados.

1. **Administración y planificación del procesador**

   - **Multiplexación de la carga de trabajo:** múltiples programas desean ejecutarse "al mismo tiempo", pero el procesador (CPU) es un recurso único y limitado. La multiplexación es la técnica que permite que varios procesos se ejecuten de forma intercalada en el tiempo. El SO divide el tiempo disponible del procesador en pequeños intervalos (time slices o quantum) y le asigna a cada proceso su turno para ejecutarse. Cuando el tiempo asignado a un proceso se agota, el SO suspende su ejecución, guarda su estado (cambio de contexto), y da turno a otro proceso. De esta forma, los usuarios tienen la ilusión de que todos los programas se ejecutan simultáneamente, cuando en realidad el proceso cambia rápidamente entre ellos.

   - **Imparcialidad y justicia en la ejecución (fairness):** el SO debe garantizar que todos los procesos reciban una oportunidad justa de ejecutarse. Esto significa que ningún proceso debe monopolizar el procesador indefinidamente, y que el tiempo total de espera sea equitativo para todos. La imparcialidad es crucial para mantener la responsabilidad del sistema: si un proceso acapara toda la CPU, otros quedarán congelados y el sistema será inutilizable.

   - **Evitar bloqueos (deadlocks):** un deadlock es una situación donde dos o más procesos quedan bloqueados de forma permanente, esperándome mutuamente por recursos que nunca serán liberados. Por ejemplo, si el proceso A posee el recurso 1 y espera por el recurso 2, mientras que el proceso B posee el recurso 2 y espera por el recurso 1, ambos quedarán bloqueados para siempre. El SO implementa mecanismos para detectar, evitar o prevenir deadlocks, garantizando que el sistema pueda continuar progresando.

   - **Manejo de prioridades:** no todos los procesos tienen la misma importancia. El SO permite asignar prioridades a los procesos: los procesos de alta prioridad (como tarea críticas del sistema) deberían recibir más tiempo de CPU que los procesos de baja prioridad (como aplicaciones en segundo plano). El planificador del SO toma en cuenta estas prioridades al decidir qué proceso ejecutar en cada momento, permitiendo un mayor control sobre la distribución de recursos.

2. **Administración de memoria**

   La memoria es uno de los recursos más críticos y limitados de una computadora. El SO debe administrarla de manera eficiente para permitir que múltiples programas coexistan en el sistema sin interferirse mutuamente.

   - **Administración eficiente de memoria:** el SO es responsable de asignar memoria a cada proceso que se crea y de liberarla cuando el proceso termina. Pero esta tarea es compleja porque la memoria disponible es limitada y debe dividirse entre muchos procesos simultáneamente. El SO debe garantizar que cada proceso reciba la cantidad de memoria que necesita, evitando tanto la fragmentación (espacios inutilizables entre bloques asignados) como el desperdicio. Para esto implementa diferentes estrategias de asignación: puede asignar bloques contiguos de memoria, usar listas ligadas para fragmentos no contiguos, o emplear técnicas más sofisticadas como buddy system o best fit.

- **Jerarquías de memoria, memoria física vs memoria virtual: l**a **memoria física (RAM)** es la memoria real disponible en la computadora, pero es limitada y costosa. Para superar esta limitación, el SO introduce el concepto de **memoria virtual**: crea la ilusión de que cada proceso tiene acceso a un espacio de memoria mucho mayor que la RAM física disponible.

  La memoria virtual funciona en conjunto con el almacenamiento secundario (disco duro). El SO divide el espacio de direcciones virtual de cada proceso en bloques pequeños llamados **páginas** (típicamente de 4 KB). Las páginas que se están usando actualmente se almacenan en RAM, mientras que las que no se necesitan inmediatamente se guardan en el disco (en un área llamada espacio de swap). Cuando un proceso necesita acceder a una página que está en disco, el SO la trae a memoria principal (page fault), y si la memoria está llena, desaloja otra página al disco.

  Esto crea una **jerarquía de memoria**: registros (muy rápido, mínima capacidad) → cache de CPU → memoria principal RAM (rápida, limitada) → almacenamiento secundario (lento, capacidad grande). El SO, junto con el hardware, trabaja para mantener en los niveles superiores (más rápidos) los datos que se usan frecuentemente.

- **Protección de programas que compiten o se ejecutan concurrentemente: c**uando múltiples procesos se ejecutan simultáneamente, existe el riesgo de que un proceso acceda a la memoria asignada a otro, causando corrupción de datos o comportamientos impredecibles. El SO implementa **mecanismos de protección de memoria** para evitar esto.

  El principal mecanismo es el uso de **espacios de direcciones virtuales aislados**: cada proceso tiene su propio espacio de direcciones virtuales, de modo que un proceso no puede acceder directamente a la memoria de otro. Cuando un proceso intenta acceder a una dirección fuera de su rango permitido, el hardware genera una excepción (protection fault) y el SO termina el proceso o maneja el error.

  Además, el SO utiliza **registros de límite** (base y límite) o **tablas de páginas** que definen qué regiones de memoria puede acceder cada proceso. El hardware verifica estas restricciones en cada acceso a memoria, permitiendo que el SO mantenga el aislamiento incluso cuando múltiples procesos se ejecutan intercaladamente en la CPU.

3. **Administración del almacenamiento, sistema de archivos**

   El SO debe gestionar no solo la memoria principal, sino también el almacenamiento secundario y los dispositivos periféricos. Esta administración es fundamental para la persistencia de datos y la comunicación con el usuario.

   - **Acceso a medios de almacenamiento externos:** el SO permite que los procesos y usuarios accedan a diferentes medios de almacenamiento externo: discos duros internos, unidades SSD, memorias USB, CD/DVD, unidades de red, etc. El SO proporciona una interfaz uniforme para acceder a todos estos medios, independientemente del tipo físico. Por ejemplo, un programa puede leer un archivo de un disco local o de una unidad USB usando el mismo conjunto de instrucciones, sin necesidad de saber cuál es el medio físico subyacente.

   - **Administración de dispositivos:** los dispositivos periféricos (impresoras, escáneres, monitores, teclados, ratones, etc.) son componentes hardware que requieren control especializado. El SO actúa como intermediario entre los programas de aplicación y estos dispositivos.

   - **Ocultamiento de dependencias de hardware:** cada dispositivo tiene características únicas y protocolos específicos de comunicación. El SO oculta esta complejidad proporcionando **controladores (drivers)** que traducen comandos genéricos en instrucciones específicas para cada dispositivo. Por ejemplo, cuando un programa quiere imprimir un documento, emite un comando genérico "imprimir". El SO, mediante el driver de la impresora, traduce este comando en la secuencia exacta de instrucciones que entiende esa impresora específica. Esto permite que los programas sean portables entre diferentes máquinas con diferentes dispositivos.

   - **Administración de accesos simultáneos:** múltiples procesos pueden desear acceder al mismo dispositivo al mismo tiempo. El SO debe coordinar estos accesos para evitar conflictos. Por ejemplo, si dos programas

intentan imprimir simultáneamente, el SO coloca los trabajos en una cola y los atiende uno a uno, asegurando que cada uno se complete correctamente. El SO también debe manejar las prioridades: un trabajo de impresión urgente puede adelantarse en la cola si es necesario.

4. **Detección de errores y respuestas**

   Durante la ejecución de un sistema, constantemente pueden ocurrir errores de diversa índole. El SO debe detectar estos errores, analizarlos y responder de manera apropiada para mantener la estabilidad y confiabilidad del sistema.

   - **Errores de memoria/ CPU:** la memoria RAM puede experimentar errores (bit flips) donde un bit cambia espontáneamente de 0 a 1 o viceversa, generalmente por radiación cósmica o problemas eléctricos. El CPU también puede generar errores de cálculo o fallos en sus circuitos. Algunos sistemas implementan **ECC (Error Correcting Code)** que permite detectar y corregir automáticamente errores de un bit, o al menos detectar errores de múltiples bits para que el SO pueda reaccionar. Cuando el SO detecta un error de memoria crítico, puede reinicios el sistema, fallar una operación específica, o registrar el evento en un log para diagnóstico posterior.
   - **Errores de dispositivos:** los dispositivos periféricos (discos duros, impresoras, interfaces de red, etc.) también pueden fallar. Un disco duro puede tener sectores dañados (bad sectors), una impresora puede desconectarse inesperadamente, o una interfaz de red puede perder conectividad. El SO debe detectar estos errores y notificarlo al usuario o aplicación afectada.
   - **Errores aritméticos:** un programa puede intentar operaciones inválidas como división por cero, overflow (cuando el resultado es demasiado grande para el tipo de dato), o underflow (cuando el resultado es demasiado pequeño). Cuando el hardware detecta estas operaciones, genera una excepción (trap) que el SO intercepta. El SO puede terminar el proceso que causó el error, mostrar un mensaje de error, o permitir que el programa maneje el error si tiene código preparado para ello.
   - **Acceso no permitido a direcciones de memoria:** un programa intenta leer o escribir en una dirección de memoria que no le pertenece (fuera de su espacio de direcciones virtual permitido). Esta es una violación grave de protección que el SO debe evitar. El hardware genera una excepción de protección (protection fault o segmentation fault), y el SO típicamente termina el proceso ofensor para proteger la integridad del sistema y otros procesos.
   - **Incapacidad del SO para conceder una solicitud de una aplicación:** a veces una aplicación solicita al SO algo que es imposible de realizar.

5. **Interacción del usuario con shell**
6. **Contabilidad**
   - Recoger estadísticas del uso.
   - Monitorear parámetros de rendimiento.
   - Anticipar necesidades de mejoras futuras.
   - Dar elementos si es necesario facturar tiempo de procesamiento.

Funciones principales de un SO

- Brindar abstracciones de alto nivel a los procesos de usuario.
- Administrar eficientemente el uso de la CPU.
- Administrar eficientemente el uso de la memoria.
- Brindar asistencia al proceso de E/S por parte de los procesos.

Apoyo del hardware

El So no puede funcionar solo; depende del hardware para implementar sus funciones de control y protección. El hardware proporciona tres mecanismos fundamentales.

1. **Modos de ejecución:** el procesador soporta dos modos de operación distintos que definen qué instrucciones puede ejecutar un programa.

   El procesador contiene un bit especial (mode bit) en un registro de estado llamado **Program Status Word (PSW)** o **Control Register**. Este bit indica en qué modo se está ejecutando actualmente el procesador:

   - **Mode bit = 0:** Modo **Kernel (Supervisor)** - El procesador está bajo el control del SO
   - **Mode bit = 1:** Modo **Usuario** - Se está ejecutando un programa de aplicación

   El hardware verifica constantemente este bit para determinar qué operaciones están permitidas. Este mecanismo es fundamental para la seguridad y estabilidad del sistema.
   Existen ciertas instrucciones que son **peligrosas y críticas** para el funcionamiento del sistema. Solo el SO debe poder ejecutarlas. Estas se llaman **instrucciones privilegiadas** e incluyen

   - **Cambiar el mode bit:** Solo el SO puede cambiar entre modo Kernel y modo Usuario
   - **Modificar registros de protección de memoria:** Solo el SO puede definir límites de memoria para procesos
   - **Deshabilitar/habilitar interrupciones:** Solo el SO puede controlar cuándo se aceptan interrupciones
   - **Acceder a dispositivos de E/S:** Solo el SO puede comunicarse directamente con impresoras, discos, redes, etc.
   - **Realizar operaciones de halt o reset:** Solo el SO puede detener o reiniciar el sistema.

   Si un programa de usuario intenta ejecutar una instrucción privilegiada, el hardware **genera automáticamente una excepción** (illegal instruction exception). El SO intercepta esta excepción, típicamente termina el proceso que la causó, y continúa ejecutando otros procesos. Esto evita que un programa malicioso o defectuoso pueda dañar el sistema.

   Cuando el SO necesita acceder a estructuras internas (como tablas de procesos, tablas de memoria, etc.) o ejecutar código que no pertenece al proceso actual (como rutinas del kernel), **debe estar en modo Kernel** porque solo en ese modo tiene los permisos necesarios.

   Modo Usuario y aislamiento de direcciones

   Cuando se ejecuta un programa de aplicación en modo Usuario, el hardware impone restricciones severas para proteger el sistema:

   - **Aislamiento de memoria:** Cada proceso tiene su propio **espacio de direcciones virtual** aislado. Un proceso en modo Usuario solo puede acceder a las direcciones que le pertenecen (aquellas dentro de su rango permitido definido por los registros base y límite).
   - **Si intenta acceder fuera:** Cuando un proceso intenta leer o escribir en una dirección fuera de su rango permitido, el **hardware detecta inmediatamente la violación** y genera una excepción (segmentation fault o protection fault). El SO intercepta esta excepción y típicamente termina el proceso.
   - **No acceso directo a hardware:** Un proceso en modo Usuario no puede ejecutar instrucciones de E/S, no puede cambiar registros de memoria, no puede acceder a dispositivos. Para hacer cualquier cosa de esto, **debe solicitar al SO** mediante una **llamada al sistema (system call)**.

   Cuando se arranque el sistema, arranque con el bit en modo supervisor y cada vez que comienza a ejecutarse un proceso de usuario, este bit se debe poner en modo usuario mediante una instrucción especial. Cuando hay un trap o una interrupción, el bit de modo se pone en modo kernel.

   Cuando el proceso de usuario intenta por sí mismo ejecutar instrucciones que pueden causar problemas, el HW lo detecta como una operación ilegal y produce un trap al SO.

2. **Interrupción de clock:** el hardware incluye un reloj que genera interrupciones periódicamente. Cuando ocurre una interrupción de clock, el control pasa al SO aunque un proceso estuviera ejecutándose en modo usuario. Permite que el SO **recupere el control de la CPU** regularmente. Sin esto, un proceso podría acaparar el procesador indefinidamente. Con el timer, el SO puede
   - Suspender el proceso actual.
   - Ejecutar otro proceso que está esperando.
   - Garantizar que todos los procesos obtengan tiempo de CPU.
3. **Protección de al memoria: e**l SO utiliza dos registros del hardware para delimitar el espacio de direcciones de cada proceso:
   - **Registro Base:** Dirección inicial permitida
   - **Registro Límite:** Tamaño máximo permitido

   Solo el SO en modo Kernel puede modificar estos registros. Cuando ocurre un cambio de contexto, el SO actualiza estos valores para el nuevo proceso.

   Cada vez que un proceso accede a memoria, el hardware comprueba automáticamente si la dirección está dentro del rango permitido (base ≤ dirección < base + límite). Si la dirección es válida, se convierte la dirección lógica a física usando el registro de reubicación y se completa el acceso. Si es inválida, el hardware genera una excepción (addressing error) y el SO termina el proceso.

   De esta forma, cada proceso está completamente aislado: no puede acceder a la memoria del SO ni de otros procesos. El hardware valida cada acceso en tiempo real sin intervención del SO, garantizando protección rápida y confiable.
4. **Protección de la E/S:** las instrucciones de E/S son aquellas que controlan la comunicación con dispositivos periféricos (disco, impresora, red, etc.). Estas instrucciones son **privilegiadas**, lo que significa que solo pueden ejecutarse en modo Kernel.

   Un proceso en modo Usuario no puede ejecutar directamente instrucciones de E/S. Si lo intenta, el hardware genera una excepción (illegal instruction) y el SO termina el proceso. Esto evita que un programa malicioso o defectuoso acceda directamente a dispositivos y los dañe o interfiera con otras operaciones de E/S.

   ¿Cómo realiza E/S un proceso usuario entonces? El proceso debe solicitar al SO mediante una **llamada al sistema (system call)** indicando qué operación de E/S desea realizar (ej: "leer archivo", "imprimir documento"). El SO valida la solicitud (¿tiene permisos? ¿el dispositivo está disponible?), realiza la operación de E/S en modo Kernel usando las instrucciones privilegiadas, y luego devuelve el resultado al proceso.

   De esta forma, el SO controla y coordina todo acceso a dispositivos, garantizando que:
   - Solo procesos autorizados accedan a ciertos dispositivos.
   - Las operaciones de E/S no interfieran entre sí.
   - Los dispositivos se usen de manera ordenada y eficiente.
5. **Protección de la CPU:** usa la interrupción por clock para evitar que un proceso se apropie de la CPU. El kernel le da valor al contador que se decrementa con cada tick de reloj y al llegar a cero, puede expulsar al proceso para ejecutar otro.

System calls

Mecanismo por el cual los programas de usuario solicitan servicios al SO. Es la única forma permitida para que un proceso en modo usuario acceda a recursos protegidos (hardware, memoria del kernel, etc).

Un programa usuario invoca una función de biblioteca que internamente ejecuta una instrucción especial (TRAP, INT, SYSCALL, según la arquitectura). Esta instrucción cambia automáticamente el mode bit a Kernel y transfiere el control a una rutina del SO, así el SO

1. Valida la solicitud.
2. Realiza la operación solicitada en modo kernel.
3. Vuelve a cambiar a modo usuario.

Los parámetros de la system call se pasan al SO de varias formas

- Registros: los parámetros se colocan en registros de la CPU, siendo rápido pero limitado.
- Bloques o tablas en memoria: se crea un bloque con los parámetros y se pasa la dirección al SO, permitiendo muchos parámetros.
- Pila: los parámetros se almacenan en la pila.

Las system calls se ejecutan en modo kernel, permitiendo al SO

- Acceder a memoria protegida.
- Ejecutar instrucciones privilegiadas.
- Controlar dispositivos.
- Acceder a estructuras internas del kernel.

Las system calls se pueden agrupar según el tipo de servicio que proporcionan

- Control de procesos
  - fork() crea un nuevo proceso.
  - exec() reemplaza el código del proceso actual con otro programa.
  - exit() termina el proceso actual.
  - wait() espera a que un proceso hijo termine.
  - kill() termina otro proceso.
- Manejo de archivos
  - open() abre un archivo.
  - close() cierra un archivo.
  - read() leer datos de un archivo.
  - write() escribir datos en un archivo.
  - mkdir() crear un directorio.
  - delete() eliminar un archivo.
- Manejo de dispositivos
  - ioctl() controlar dispositivos.
  - read() / write() leer/ escribir desde dispositivos.
  - seek() posicionarse en un dispositivo.

En resumen, los system calls son servicios agrupados en categorías según su función. Cada categoría proporciona herramientas para que los programas usuario accedan a recursos protegidos del sistema controladamente.

Tipos de kernel: monolítico

Un **kernel monolítico** es una arquitectura donde todo el código del SO se compila en un único programa grande que se ejecuta como una unidad en modo kernel.

La arquitectura se organiza en capas jerárquicas

- Software: en la capa superior están los programas de usuario que solicitan servicios al SO mediante system calls.
- Kernel: una única estructura grande que contiene todo el código del SO. Incluye los módulos para gestionar procesos, memoria, archivos, dispositivos, sincronización, etc. Todo este código se ejecuta en modo kernel con acceso directo al hardware.
- Hardware: en la capa inferior está el hardware físico que el kernel controla.

Monolítico vs microkernel

En un kernel monolítico, **todo el código del SO está en modo kernel** como un único bloque. Cuando la aplicación hace una system call

1. La aplicación (en modo usuario) realiza una system call.
2. El control pasa al kernel monolítico.
3. El kernel accede directamente a todos los módulos.
4. El kernel realiza la operación y devuelve el control a la aplicación.

Es muy rápido porque no hay cambio de contexto entre componentes, la desventaja es que un error en cualquier módulo puede romper todo el sistema.

En un microkernel, **solo las funciones críticas están en modo kernel.** El resto de los servicios, se ejecutan en modo usuario como procesos servidores.

Cuando una aplicación necesita un servicio

1. La aplicación en modo usuario hace una system call.
2. El control pasa al microkernel.
3. El microkernel solo proporciona algunos servicios: IPC (inter-process communitacion), virtual memory básica y scheduling.
4. El microkernel comunica a la aplicación con el servidor apropiado usando IPC.
5. El servidor en modo usuario realiza la operación.
6. El resultado vuelve a través del microkernel a la aplicación.

Lo bueno es que es más modular y seguro. Si un servidor falla, no rompe todo el sistema, lo malo es que es más lento porque hay múltiples cambios de contexto.

Evolución de un SO

Los SO evolucionan con el objetivo de

- Soportar nuevos tipos de hardware.
- Brindar nuevos servicios.
- Ofrecer mejoras y alternativas a problemas existentes en planificación, manejo de memoria, etc.

En la década del 50/60 no existían los SO si no que un operador cargaba manualmente un programa, se introducía en una máquina y se presionaban botones e interruptores en la consola. Cuando se terminaba, otro operador cargaba el siguiente programa. Por lo tanto los programas se ejecutaban secuencialmente sin ningún software de control intermediario.

Esto requería alto nivel de especialización y coordinación por lo cual era difícil de planificar y costoso además de mucho tiempo muerto entre programas. El proceso era

1. Cargar el compilador: se insertaban las tarjetas del compilador en la máquina.
2. Cargar el código fuente: insertar las tarjetas con el programa a compilar.

3. Guardar el programa compilado: almacenarlo en cinta magnética o tarjetas.
4. Cargar el programa compilado: insertar nuevamente las tarjetas/ cintas.
5. Ejecutar el programa: finalmente correr el programa.
6. Linkeo: vincular librerías necesarias.

Cada paso requería intervención manual, configuración física y cambio de medios de almacenamiento.

Estos problemas llevaron a la creación de los primeros SO que automatizaron la transición entre programas, optimización del uso de recursos y eliminar la necesidad de intervención manual constante.

Sistemas por lotes (batch)

El primer software de control fue el monitor residente, un programa pequeño que residía permanentemente en memoria. Era el primer SO primitivo. Su función era automatizar la transición entre programas.

El monitor residente controlaba la ejecución de una secuencia de trabajos que se cargaban juntos en la máquina. En lugar de cargar un programa, ejecutarlo manualmente, descargarlo, y repetir, ahora se preparaba un "lote" (batch) con varios trabajos de una vez.

Los trabajos se colocaban juntos en cintas magnéticas o tarjetas perforadas, formando un lote que se procesaba automáticamente de principio a fin. Cuando un programa terminaba su ejecución, el control volvía automáticamente al Monitor Residente, que cargaba y ejecutaba el siguiente programa del lote. No había intervención manual entre trabajos.

Ventajas

- Automatización: Eliminó la necesidad de intervención manual entre programas
- Eficiencia: Redujo el tiempo ocioso de la máquina
- Costos: Mejor utilización de un recurso muy costoso

Desventajas

- No-interactivo: Los usuarios no podían interactuar con sus programas mientras se ejecutaban
- Esperas largas: Los errores solo se descubrían después, cuando se entregaban los resultados
- Inflexible: Era difícil agregar o modificar trabajos una vez iniciado el lote

Sistema Batch

El problema fundamental de los sistemas batch es que la **CPU permanece ociosa (idle) la mayor parte del tiempo**, lo que resulta en una mala utilización de un recurso muy costoso.

Los dispositivos de E/S son órdenes de magnitud más lentos que el procesador, cuando un programa ejecuta una instrucción de E/S

1. La CPU envía la solicitud de E/S al dispositivo.
2. El procesador se queda esperando hasta que el dispositivo completa la operación.
3. Durante este tiempo muerto, la CPU no puede hacer nada útil.
4. Cuando finalmente completa la E/S, el programa continúa.

Este problema llevó a la introducción de la **multiprogramación**: mantener múltiples programas en memoria simultáneamente. Cuando un programa está esperando E/S, la CPU puede cambiar a ejecutar otro programa, manteniendo el procesador ocupado.

Multiprogramación

La clave para mejorar la eficiencia del sistema batch fue el **spooling.** En lugar de esperar a que un programa complete toda su ejecución antes de cargar el siguiente, el SO ahora carga múltiples tareas en **almacenamiento secundario** mientras otras se ejecutan.

Mientras un programa A está ejecutándose en la CPU y realiza E/S, el SO aprovecha ese tiempo muerto para cargar en disco el siguiente programa B desde la entrada estándar. De esta forma, cuando A termina, B ya está listo en disco para ejecutarse inmediatamente. No hay tiempo muerto esperando a que llegue el siguiente trabajo.

Job scheduling (planificación de trabajos)

Con múltiples tareas almacenadas en disco, el SO ya no necesita ejecutarlas en el orden en que fueron cargadas originalmente. El SO puede elegir inteligentemente cuál tarea ejecutar a continuación basándose en criterios como prioridad del trabajo, tiempo estimado de ejecución, recursos necesarios, disponibilidad de recursos.

Multiprogramación en memoria

El avance crucial fue que el SO ahora mantiene varias tareas en memoria principal simultáneamente, no solo una. Cuando una tarea realiza E/S y se bloquea esperando

1. La CPU no permanece ociosa.
2. El SO cambia de contexto y ejecuta otra tarea que está lista.
3. Mientras esa tarea espera E/S, cambia a una tercera.
4. De esta forma, la CPU está constantemente ejecutando algo útil.

La secuencia de programas es de acuerdo a prioridad u orden de llegada, cuando el proceso necesita realizar una operación de E/S, la CPU en lugar de permanecer ociosa, es usada para otro momento. Después de que se completa la atención de la interrupción, el control puede o no retornar al programa que se establece ejecutando al momento de la interrupción.

Tiempo compartido

Utiliza la multiprogramación para manejar múltiples trabajos interactivos donde el tiempo del procesador es compartido entre múltiples trabajos así, múltiples usuarios podrían acceder simultáneamente al sistema usando terminales.

Los procesos usan la CPU por un periodo máximo de tiempo, luego del cual se le da la CPU a otro proceso.

Clase 2

Los procesos son un programa en ejecución, llamados también tarea o job.

Programa vs proceso

Un programa es una **entidad estática,** es el código fuente compilado, almacenado en disco como un archivo ejecutable, caracterizado por

- Ser estático, no cambia por sí solo. Es solo un conjunto de instrucciones estrictas en código que permanencen iguales en el disco.
- No tiene program counter, el PC es un registro que indica cuál es la siguiente instrucción a ejecutar.
- Ciclo de vida: existe desde que se crea/ edita hasta que se borra del disco.

Un proceso es una **entidad dinámica**

- Es dinámico, cambia constantemente mientras se ejecuta. El estado del proceso (variables, registros, stack, etc) se modifica a cada instrucción.
- Tiene program counter.
- Comienza cuando se dispara y termina cuando finaliza la ejecución.

Un programa puede generar múltiples procesos, el mismo ejecutable en disco puede lanzarse varias veces simultáneamente mientras que un proceso requiere un programa ya que todo proceso debe ejecutar el código de algún programa.

Componentes de un proceso

Un proceso para poder ejecutarse incluye como mínimo

1. Sección de código.
2. Sección de datos, que son las variables globales.
3. Stacks (datos temporarios como parámetros, variables temporales y direcciones de retorno).

   Un proceso típicamente cuenta con uno o más stacks

   - Stack de usuario: se usa cuando el proceso ejecuta código en modo usuario.
   - Stack de kernel: se usa cuando el proceso entra en modo kernel, ejecutando una system call o manejando una excepción.

   Estos stacks están separados para mantener el aislamiento entre modo usuario y modo kernel. Los stacks se crean automáticamente cuando se crea el proceso. Su tamaño no es fijo ya que se ajusta dinámicamente en runtime según sea necesario. Cuando se necesita más espacio en pila, el SO lo proporciona automáticamente.

   El stack está formado por stack frames (marcos de pila) que se agregan y se retiran dinámicamente.

   1. Push: cuando se llama a una función/ rutina, se crea un nuevo stack frame y se empuja al tope del stack.
   2. Pop: cuando la función retorna, ese stack frame se extrae del stack y se destruye.

   Cada stack frame contiene

   - Parámetros de la rutina: los argumentos pasados a la función que se acaba de llamar.
   - Variables locales: variables declaradas dentro de la función.
   - Dirección de retorno: la dirección de la instrucción que debe ejecutarse cuando la función termina.
   - Stack pointer anterior: el valor del stack pointer antes de entrar a esa función.

Atributos de un proceso

- Identificación del proceso y del proceso padre.
- Identificación del usuario que lo “disparó”.
- Si hay estructuras de grupos, grupo que lo disparó.
- En ambientes multiusuario, desde qué terminal y quién lo ejecutó.

Process control block (PCB)

Estructura de datos asociada al proceso, será lo primero que se crea cuando se crea un proceso y lo último que se borra cuando termina.

Contendrá información asociada con cada proceso

- PID, PPID, etc.

- Valores de los registros de la CPU.
- Planificación (estado, propiedad, tiempo consumido, etc).
- Ubicación en memoria.
- Accounting.
- Entrada/ salida.

Espacio de direcciones

El espacio de direcciones de un proceso es el conjunto de todas las direcciones de memoria que el proceso puede acceder. Es como el “territorio” de memoria que le pertenece al proceso, el cual incluirá

- Stack: la pila donde se almacena variables locales, parámetros de funciones y direcciones de retorno.
- Heap: la región de memoria dinámica donde se asignan bloques.
- Datos: variables globales y estáticas.
- Text: el código del programa.

El espacio de direcciones NO incluye el PCB ni las estructuras de control del SO asociadas al proceso. El PCB es una estructura del kernel que contiene metadatos del proceso. Está en el espacio de memoria del kernel, no en el espacio del proceso usuario.

Cuando un proceso se ejecuta en modo usuario, está confinado a su propio espacio de direcciones. Solo puede acceder a direcciones dentro de su rango asignado, no puede acceder a la memoria de otros procesos ni acceder a la memoria del kernel, en el caso de que acceda a un espacio fuera de su rango, el hardware lanzará una excepción.

Este aislamiento es fundamental para la protección y estabilidad del sistema.

Cuando un proceso entra en modo kernel, el SO puede

- Acceder a las estructuras internas del proceso para leer/modificar su estado, prioridad, recursos.
- Acceder a espacios de direcciones de otros procesos.
- Acceder a memoria del kernel mismo.

Contexto de un proceso

Incluye toda la información que el SO necesita para administrar el proceso y la CPU para ejecutarlo correctamente. Estará formado por los registros de CPU, inclusive el PC, prioridad del proceso, E/S pendientes, etc.

Cambio de contexto

Se produce cuando la CPU cambia de un proceso a otro, resguardando el contexto del proceso saliente que pasa a espera y retornará después de la CPU.

Se debe cargar el contexto del nuevo proceso y comenzar desde la instrucción siguiente a la última ejecutada en dicho contexto.

Será tiempo no productivo de la CPU y el tiempo que consume depende del HW.

Kernel del SO

El kernel es un conjunto de módulos de software que se ejecuta en el procesador como cualquier otro proceso generando la pregunta de si el kernel es un proceso y quién lo controla, hay dos enfoques conceptuales

Enfoque 1: kernel como entidad independiente

El kernel se ejecuta por fuera de todo proceso, como una entidad separada en modo privilegiado

- El kernel tiene su propia región de memoria independiente.

- El kernel tiene su propio stack para sus operaciones.
- Cuando el SO finaliza su actividad, devuelve el control al proceso.

El kernel NO es un proceso. El concepto de proceso solo aplica a programas de usuario. El kernel es una entidad independiente en modo supervisor/ privilegiado.

Enfoque 2: el kernel "dentro" del proceso

El código del kernel reside dentro del espacio de direcciones de cada proceso y se ejecuta en el MISMO contexto que el proceso

- El código del kernel está distribuido en cada espacio de direcciones de proceso.
- Se puede ver como una colección de rutinas que el proceso utiliza.
- Cada proceso tiene su propio user stack y kernel stack.
- Cuando ocurre una interrupción o system call, el proceso cambia de modo sin cambiar de contexto completo.
- El kernel es compartido por todos los procesos.
- El proceso es el que se ejecuta en modo usuario y su kernel del So en modo kernel.

Cuando el SO atiende una interrupción (incluyendo system calls), se ejecuta **en el contexto del proceso que se encontraba en ejecución**, pero en modo Kernel. Es muy eficiente porque no requiere un cambio de contexto completo.

En resumen:

- **Enfoque 1** (histórico, primeros SO): Kernel como supervisor externo
- **Enfoque 2** (moderno, actual): Kernel integrado en el contexto de cada proceso, accesible mediante cambio de modo

Estados de un proceso

En su ciclo de vida, un proceso pasa por diferentes estados

- **Nuevo (new):** un usuario dispara el proceso el cual es creado por otro proceso, su padre. En este estado se crean las estructuras asociadas y el proceso queda en la *cola de procesos*, normalmente en espera de ser cargado en memoria.
- **Listo para ejecutar (ready):** luego que el scheduller de largo plazo eligió al proceso para cargarlo en memoria, el proceso queda en estado listo. El proceso solo necesita que se le asigne CPU y quedará en la cola de proesos listos.
- **Ejecutándose (running):** el scheduler de corto plazo lo eligió para asignarle CPU. Tendrá la CPU hasta que se termine el período de tiempo asignado (quantum o time slice), termine o hasta que necesite realizar una operación E/S.
- **En espera (waiting):** el proceso necesita que se cumpla el evento esperado para continuar. El evento puede ser la terminación de una E/S solicitada o la llegada de una señal por parte de otro proceso. Sigue en memoria pero no tiene la CPU, cuando se cumpla el evento, pasará al estado de listo.
- Terminado (terminated).

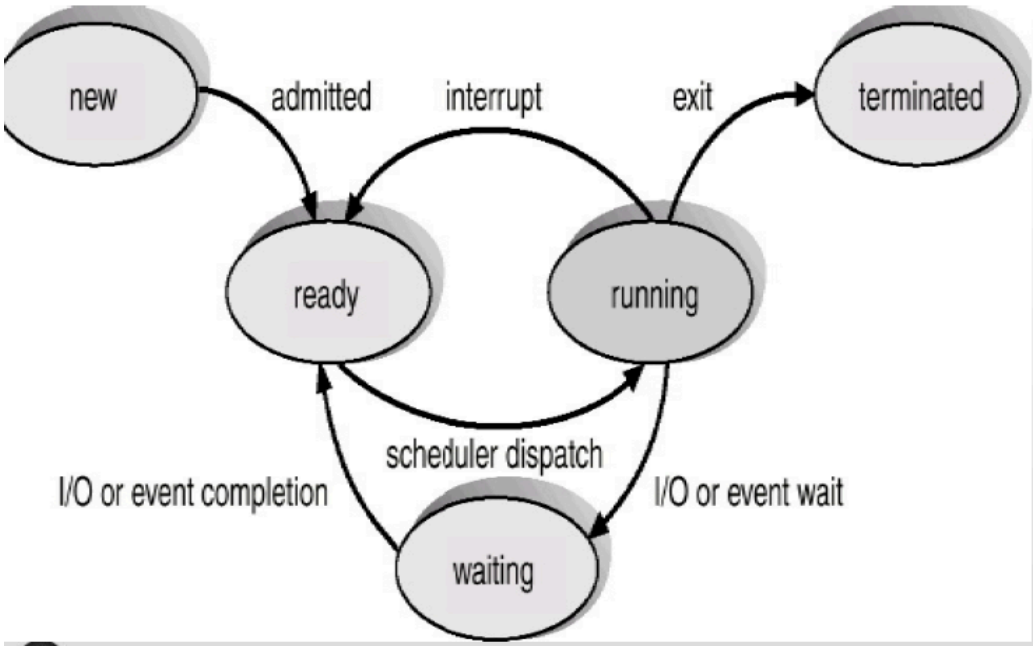
Transiciones

- New → ready: por elección del scheduler de largo plazo.
- Ready → running: por elección del shceduler de corto plazo (asignará la CPU).

- Running → waiting: el proceso “se pone a dormir”, esperando a un evento.
- Waiting → ready: terminó la espera y compite nuevamente por la CPU.
- Running → ready: es un caso especial que cuando el proceso termina su quantum (franja de tiempo) sin haber necesitado ser interrumpido por un evento, pasa al estado de ready para competir por CPU. Se trata de un caso distinto de los anteriores porque el proceso es expulsado de la CPU contra su voluntad, ocurriendo en algoritmos apropiativos.

Explicación por estado

1. Ejecución en modo usuario.
2. Ejecución en modo kernel.
3. El proceso está listo para ser ejecutado cuando sea elegido.
4. Proceso en espera en memoria principal.
5. Proceso listo, pero el swapper debe llevar al proceso a memoria principal antes que el kernel lo pueda elegir para ejecutar.
6. Proceso en espera en memoria secundaria.
7. Proceso retornado desde el modo kernel al user. Pero el kernel se apropia, hace un context switch para darle la CPU a otro proceso.
8. Proceso recientemente creado y en transición: existe, pero aún no está listo para ejecutar, ni está dormido.
9. El proceso ejecutó la system call exit y está en estado zombie. Ya no existe más, pero se registran datos sobre su uso, codigo resultante del exit. Es el estado final.



Colas en la planificación de procesos

Para realizar la planificación, el SO usa la PCB de cada proceso como abstracción del mismo, las PCB se enlazan en colas siguiente un orden determinado.

Algunas son

- Cola de trabajos o procesos: contiene todas las PCB de procesos en el sistema.
- Colas de procesos listos: PCB de procesos residentes en memoria principal esperando para ejecutarse.

Módulos de la planificación

Son módulos del kernel que realizan distintas tareas asociadas a la planificación que se ejecutan ante determinados eventos que así lo requieren como creación/ terminación de procesos, eventos de sincronización o de E/S, finalización de lapso de tiempo, etc.

Hay distintos módulos de planificación

- **Scheduler de long term**: decide cuál proceso cargar en memoria desde disco. Controla el grado de multiprogramación, es decir, la cantidad de procesos en memoria. Puede no existir este scheduler y que la tarea sea hecha por el short term.
- **Scheduler de short term**: decide cuál proceso en memoria ejecutar en la CPU. Decide a cuál de los procesos de la cola de listos se elige para que use la CPU.
- **Scheduler de medium term**: si es necesario reduce el grado de multiprogramación sacando temporalmente de memoria los procesos que sea necesario para mantener el equilibrio del sistema.

El dispatcher y loader son módulos que implementan las decisiones de estos schedulers.

- Dispacher: es el módulo responsable de ejecutar la decisión del short term scheduler. Su función es realizar el cambio de contexto y poner en ejecución el proceso elegido.
  1. Guarda el estado del p[roceso actual.
  2. Cambia del modo usuario al modo kernel.
  3. Salta a la instrucción del siguiente proceso que debe ejecutarse.

  Es decir, el dispatcher salta el control de ejecución al proceso elegido por el scheduler, restaurando su contexto anterior.
- Loader: módulo responsable de ejecutar la decisión del long term scheduler. Su función es cargar en memoria principal el proceso elegido por el long term scheduler.
  1. Busca el programa en disco.
  2. Lee el ejecutable del disco.
  3. Crea un espacio de direcciones en memoria RAM.
  4. Copia el código y datos del programa en memoria.
  5. Inicializa las estructuras de datos del proceso como PCB, stack, etc.
  6. Marca el proceso como ready para ser ejecutado.

  Una vez que el loader termina, el proceso está en memoria y dispone para que el short term scheduler lo elija y el dispatcher lo ejecute.

El dispatcher y loader pueden no existir como módulos separados explícitamente en todos los SO pero sus funciones siempre deben cumplirse: cargar procesos en memoria, cambiar el contexto y ejecutar procesos.

Pueden no existir como módulos separados de los schedulers vistos pero la función debe cumplirse.

Comportamiento de los procesos

Una ráfaga es un período continuo de tiempo durante el cual un proceso realiza una actividad específica sin cambios. Es un intervalo ininterrumpido de trabajo. Los proceso alternan constantemente entre **ráfagas de CPU y ráfagas de E/S.**

- **CPU burst**: el proceso ejecuta instrucciones en el procesador.

- **I/O burst**: el proceso espera por operaciones de E/S.

Existen dos tipos de procesos

- CPU-bound: pasan la mayor parte del tiempo usando la CPU.
- I/O/bound: pasan la mayor parte del tiempo esperando E/S.

Planificación

Es la tarea de decidir cuál proceso de los “listos” se ejecutará a continuación en un sistema multiprogramado mientras que un **algoritmo de planificación** es el método que usa el SO para tomar esa decisión.

Algoritmos apropiativos vs no apropiativos

Los algoritmos **apropiativos** son aquellos donde el SO puede expulsar un proceso de la CPU en cualquier momento, esto ocurre cuando el quantum expira o llega un proceso de mayor prioridad.

La apropiación está relacionada al recurso CPU. En los algoritmos apropiativos (preemtive) existen situaciones que hacen que el proceso en ejecución sea expulsado, por el planificador de corto plazo, de la CPU. Estos algoritmos serán **round robin, SRTF** y **prioridades apropiativo.**

El proceso puede ser expulsado de la CPU según la planificación implementad: se termina su quantum, llega a la cola de listos un proceso de mayor prioridad o llega a la cola de listos un proceso con menor tiempo restante.

Mientras que los **no apropiativos** ejecuta un proceso hasta que voluntariamente abandone la CPU por bloqueo por E/S o termina. No hay decisiones de planificación en interrupciones de reloj. Los procesos se ejecutan hasta que el mismo, por su propia cuenta) abandone la CPU.

El proceso deja el estado de ejecución solo cuando termina, se bloquea voluntariamente o solicita una operación de E/S. Serán los **FCFC, SJF** y **prioridades no apropiativo.**

Categorías según el ambiente

El SO requiere diferentes algoritmos según el tipo de procesos.

- **Procesos batch:** no hay usuarios esperando respuesta, tendrá algoritmos no apropiativps (FCFS, SJF) con el objetivo de rendimiento, tiempo de retorno y uso de CPU.
- **Procesos interactivos:** los usuarios esperan respuesta rápida, usando los algoritmos apropiativos como round robin, prioridades, SRTF.
- **Procesos tiempo real:** plazos estrictos.

Política vs mecanismo

- Mecanismo: lo que implementa el kernel.
- Política: cómo se usan los parámetros del mecanismo (ej: qué prioridad asignar a cada proceso).

Creación de procesos

Un proceso es creado por otro proceso el cual el padre tiene uno o más procesos hijos, formando un árbol.

A la hora de crearlos,

- Se crea la PCB.
- Asignar PID (process identification).
- Se asigna memoria para regiones como stack, text y datos.
- Crea estructuras de datos asociadas.

Respecto a los procesos padres e hijo, el padre puede continuar ejecutándose concurrentemente con su hijo, el cual puede esperar a que el proceso hijo terminen para continuar la ejecución.

Los SO tienen distintos mecanismos para crear procesos, los dos principales son Unix y Windows.

- Unix utiliza un modelo de dos pasos para crear procesos
  1. **fork(), duplicación del proceso,** los system call fork() crea un nuevo proceso que es una copia exacta del proceso actual.

     Se crea un nuevo PCB para el hijo, copia el espacio de direcciones completo del padre al hijo, el proceso hijo es idéntico al padre de ese momento y ambos procesos continúan ejecutando desde el mismo punto.
  2. **exec(), reemplazo del espacio de direcciones,** la system call exec() reemplaza el espacio de direcciones del proceso actual con un nuevo programa.

     El proceso mantiene su PID y PCB, el espacio de direcciones se reemplaza completamente con el nuevo programa y comienza a ejecutarse desde su punto de entrada.
- Windows utiliza un modelo de un solo paso
  1. createProcess(), creación e inmediata recarga, la system call createProcess() crea un nuevo proceso Y carga automáticamente el programa en un solo paso

     Se crea un nuevo PCB y un espacio de direcciones y carga el programa especificado en memoria, el nuevo proceso comienza a ejecutar.

     No hay duplicación a diferencia de Unix.

Terminación de procesos

Cuando un proceso ejecuta exit(), entrega el control nuevamente al SO. El proceso padre recibe el código de retorno via wait(): el padre puede esperar a que el hijo termine y recibir un código de retorno que indica cómo terminó, `exit(0)` éxito y `exit(1)` o `exit(2)` será un error.

El proceso padre puede terminar a sus hijos por la fuerza usando `kill()` sin esperar a que terminen voluntariamente.

En resumen,

- `exit()` terminación voluntaria que retorna un código al padre.
- `wait()` el padre espera el código de retorno.
- `kill()` el padre termina forzadamente a un hijo.
- `cascada` cuando el padre termina, los hijos se terminan automáticamente.

Procesos cooperativos e independientes

- **Independiente**: el proceso no afecta ni puede ser afectado por la ejecución de otros procesos. No comparte ningún tipo de dato.
- **Cooperativo**: afecta o es afectado por la ejecución de otros procesos en el sistema. Este tipo de procesos son útiles para compartir información, acelerar el cómputo o para planificar tareas de manera tal que se puedan ejecutar en paralelo.

Clase 3

La organización y administración de la memoria principal es uno de los factores más importantes en el diseño de los SO.

Los programas y datos deben estar en el almacenamiento principal para poderlos ejecutar y referenciarlos directamente. Se espera de un SO el uso eficiente de la memoria con el fin de alojar el mayor número de procesos.

El SO debe

- Mantener un registro actualizado de cuál memora está siendo usada y cuál está libre.
- Asignar espacio cuando se necesita como cuando un nuevo proceso se crea o necesita más memoria, el SO asigna un bloque de memoria libre.
- Libera espacio de memoria asignada a procesos que han terminado.
- Lograr que el programador se abstraiga de la alocación de los programas.
- Brindar seguridad entre los procesos para que unos no accedan a secciones privadas de otros.
- Brindar la posibilidad de acceso compartido a determinadas secciones de la memoria.
- Garantizar la performance del sistema.

Administración de memoria

División lógica de la memoria

El SO divide la memoria física (RAM) en secciones lógicas para que múltiples procesos puedan coexistir sin interferirse. La RAM física es una sola unidad continua pero el SO la organiza lógicamente en particiones o espacios virtuales asignados a cada proceso.

Esto permite asegurar la protección ya que el SO aísla cada proceso en su propio espacio de direcciones lógico depende del mecanismo de cada HW.

Asignación eficiente

El objetivo es **contener** el mayor número de procesos para garantizar el mayor uso de la CPU por los mismos.

Reubicación

- El programador no debe ocuparse de conocer dónde será colocado en la memoria RAM.
- Mientras un proceso se ejecuta, puede ser sacado y traído a la memoria y posiblemente colocarse en diferentes direcciones.
- Las referencias a la memoria se deben traducir según ubicación actual del proceso.

Protección

- Los procesos no deben referenciar o acceder a direcciones de memoria de otros procesos salvo que tengan permiso.
- El chequeo se debe realizar durante la ejecución ya que no es posible anticipar todas las referencias a memoria que un proceso puede realizar.

Compartición

- Permitir que varios procesos accedan a la misma porción de memoria.
- Permite un mejor uso de la memoria RAM, evitando copias innecesarias. de instrucciones.

Espacio de direcciones

El espacio de direcciones es el rango total de direcciones de memoria que un proceso puede usar para acceder a sus instrucciones y datos.

El tamaño máximo del espacio de direcciones depende de cuántos bits usa el procesador para direcciones.

Si el procesador es de 32 bits, usa 32 bits para representar direcciones, lo cual será 2^32 direcciones diferentes.

Es importante saber que el espacio de direcciones es lógico, no físico. Las direcciones que usa un proceso no corresponden directamente a dónde está realmente en la RAM física.

Direcciones lógicas vs físicas

Una dirección lógica es una referencia a una localidad de memoria dentro del espacio de direcciones del proceso.

- Independiente de asignación actual: la dirección lógica es siempre la misma, sin importar dónde esté realmente el proceso en RAM.
- Relativa al proceso: es como un desplazamiento dentro del espacio privado del proceso.

Una dirección física es la ubicación real en la memoria RAM.

- Es donde está realmente el dato en la RAM física.
- Dirección real en el HW.
- El proceso nunca ve directamente las direcciones físicas.

Si la CPU trabaja con **direcciones lógicas**, pero la **memoria real (RAM) usa direcciones físicas**, entonces es necesario transformar dirección lógica a física. Este proceso se llama “resolución de direcciones” o address binding.

El address binding es hecho automáticamente por el HW usando tablas de página o segmentación.

Hay 3 momentos en los que las direcciones lógicas se pueden resolver en físicas

1. Tiempo de compilación: el compilador conoce exactamente dónde estarán los datos y genera direcciones físicas directamente.
2. En tiempo de carga: el loader resuelve las direcciones cuando carga el programa en memoria.
3. En tiempo de ejecución: el hardware traduce dinámicamente cada acceso a memoria.

Conversión de direcciones

Una forma simple de hacer esto es usando registros auxiliares

- Registro base: dirección de comienzo del espacio de direcciones del proceso en la RAM.
- Registro límite: dirección final del proceso o tamaño de su espacio de direcciones.

Ambos valores se fijan cuando el espacio de direcciones del proceso es cargado a memoria y varían entre procesos.

Memory management unit (MMU)

La MMU es un dispositivo de hardware que forma parte del procesador y su función es mapear direcciones lógicas a físicas.

Cuando la CPU necesita acceder a memoria, antes de llegar a la RAM física, pasa por el MMU que realiza la traducción. Este componente contiene tablas de mapeo (típicamente tablas de página) que definen para cada proceso dónde está su memoria en la RAM física. Sin el MMU, la multiprogramación tal como la conocemos no sería posible.

Reprogramar el MMU es una operación privilegiada que solo puede realizarse en modo Kernel. Cuando el SO necesita cambiar de un proceso a otro (en un context switch), debe actualizar las tablas del MMU para que apunten a la memoria del nuevo proceso.

Mecanismos de asignación de memoria

- Particiones fijas: es el primer esquema implementado donde la memoria se divide en particiones de tamaño fijo donde cada una alija un proceso y cada uno se coloca de acuerdo a algún criterio en alguna partición.
- Particiones dinámicas: es una evolución del anterior. Las particiones varían en tamaño y en número alojando en cada una un proceso y cada partición se genera dinámicamente del tamaño del proceso.

Fragmentación

Se produce cuando una localidad de memoria no puede ser utilizada por no encontrarse en forma contigua.

- Fragmentación interna: se produce en el esquema de particiones fijas y es la porción que queda sin usar.
- Fragmentación externa: se produce en el esquema de particiones dinámicas y son los huevos que van quedando en la memoria a medida que los proceso finalizan.

  Al no encontrarse en forma contigua puede darse el caso de que tengamos memoria libre para alocar un proceso pero que no la podemos usar. Para solucionar el problema se puede acudir a la compactación pero resulta costosa.

Problemas del esquema

El esquema de registro base + limite presenta algunos problemas

- Necesidad de almacenar el espacio de direcciones de forma continua en la memoria física.
- Los primeros SO definían particiones fijas de memoria y luego evolucionaron a particiones dinámicas.
- Fragmentación.
- Mantener partes del proceso que no son necesarias.
- Los esquemas de particiones fijas y fdnámicas no se usan hoy en día.

Para estos problemas se presenta la solución de paginación y segmentación.

Paginación

La paginación es un mecanismo fundamental de gestión de memoria que divide tanto la memoria física como la lógica en bloques pequeños de tamaño uniforme. La idea central es simple pero poderosa: en lugar de tratar la memoria como un continuo, la dividimos en pedazos manejables.
La memoria física, que es la RAM real de tu computadora, se divide lógicamente en pequeños trozos de igual tamaño llamados marcos (frames). Si tienes una computadora con 4 GB de RAM y usas marcos de 4 KB, entonces tienes

aproximadamente 1 millón de marcos disponibles. Cada marco es simplemente un bloque de 4 KB de RAM física consecutiva. El primero marco comienza en la dirección 0x00000000, el segundo en 0x00001000, el tercero en 0x00002000, y así sucesivamente. El SO mantiene un registro de cuál marco está ocupado y cuál está libre, de modo que sabe dónde puede asignar memoria.

De la misma manera, el espacio de direcciones lógico de cada proceso (el rango de direcciones que el proceso cree que puede usar) también se divide en trozos de igual tamaño llamados páginas. Y aquí viene lo importante: el tamaño de una página es exactamente igual al tamaño de un marco. Si los marcos son de 4 KB, las páginas también son de 4 KB. Esto es fundamental porque permite un mapeo muy elegante y eficiente. Cuando un proceso necesita acceder a una dirección lógica, el SO puede simplemente decir "esa dirección lógica está en la página número X, y esa página está en el marco número Y".

El SO mantiene una tabla de páginas para cada proceso. Esta tabla es esencialmente un diccionario que responde la pregunta: "¿En cuál marco físico está la página lógica número N?" Por ejemplo, si un proceso intenta acceder a la página 0, el SO busca en la tabla de páginas y descubre que la página 0 está en el marco 10. Si intenta acceder a la página 1, la tabla dice que está en el marco 52. Si intenta acceder a la página 3, la tabla podría indicar que esa página está actualmente en el disco (porque la RAM está llena). Cada entrada en la tabla de páginas contiene el número del marco donde reside esa página, junto con información adicional como si está válida, si está en disco o en RAM, y qué permisos tiene.

Ahora viene lo crucial: cómo el hardware interpreta una dirección lógica. Una dirección lógica consiste en dos partes. Los bits más significativos representan el número de página, y los bits menos significativos representan el desplazamiento dentro de esa página. Por ejemplo, si tienes páginas de 4 KB, eso significa que necesitas 12 bits para direccionar cualquier byte dentro de una página (porque 2 elevado a 12 es 4096). Los bits restantes forman el número de página. Entonces, si tu dirección lógica es 0x12345ABC, los primeros 20 bits (0x12345) indican cuál página, y los últimos 12 bits (0xABC) indican dónde dentro de esa página.

Cuando un proceso intenta acceder a una dirección lógica, el hardware hace lo siguiente: primero extrae el número de página y el offset de la dirección lógica. Luego busca en la tabla de páginas del proceso para encontrar en cuál marco está esa página. Una vez que conoce el número del marco, calcula la dirección física multiplicando el número del marco por el tamaño de la página y sumándole el offset. Por ejemplo, si la página 0x12345 está en el marco 74, y el tamaño de página es 4096 bytes, la dirección física sería (74 × 4096) + 0xABC. Finalmente, accede a la RAM en esa dirección física y obtiene el dato.

Lo elegante de este sistema es que es completamente transparente para el proceso. El proceso sólo conoce direcciones lógicas. No tiene idea de en qué marco físico está realmente su memoria. Dos procesos pueden estar usando exactamente la misma dirección lógica 0x1000, pero su tabla de páginas apuntará a marcos completamente diferentes, de modo que accederán a datos completamente distintos. Esto proporciona aislamiento perfecto entre procesos sin que ellos tengan que saber nada al respecto.

Además, las páginas de un proceso no necesitan estar en marcos contiguos. La página 0 podría estar en el marco 10, la página 1 en el marco 200, la página 2 en el marco 15. El SO puede colocar las páginas en cualquier marco disponible. Esto significa que no hay fragmentación externa: el espacio disponible entre procesos es siempre un múltiplo exacto del tamaño de página, no hay huecos desperdiciados.

Otro aspecto importante es que la paginación permite memoria virtual. Si la RAM se llena, el SO puede tomar una página que no se ha usado recientemente y guardarla en el disco duro. La entrada en la tabla de páginas indicaría que esa página está en disco. Cuando el proceso intenta acceder a esa página nuevamente, ocurre un "page fault", el SO trae la página del disco de vuelta a RAM (posiblemente desplazando otra página al disco), actualiza la tabla de páginas, y el proceso continúa como si nada hubiera pasado. Esto permite que los procesos usen mucha más memoria de la que hay disponible en RAM física.

En resumen, la paginación es un sistema donde cada dirección lógica se divide en número de página y offset. El SO usa una tabla de páginas para mapear cada página lógica a un marco físico. El hardware hace esta traducción automáticamente y transparentemente cada vez que el proceso accede memoria. Este mecanismo proporciona aislamiento perfecto entre procesos, permite una distribución flexible de memoria, no causa fragmentación, y es la base del sistema de memoria virtual que permite que los procesos tengan más memoria virtual que la RAM física disponible.

Segmentación

Es un esquema que se asemeja a la visión del usuario donde el programa se divide en partes/ secciones. Un programa es una colección de segmentos donde cada uno es una unidad lógica como programa principal, procedimientos y funciones, variables locales, globales, etc. Esto puede causar fragmentación.
Una dirección lógica en segmentación consiste en **dos partes**, pero son conceptualmente diferentes a las de paginación. La primera parte es el **selector de segmento**, que identifica cuál segmento estás accediendo (por ejemplo, segmento 0 para código, segmento 1 para datos, segmento 2 para heap). La segunda parte es el **desplazamiento dentro del segmento**, que indica a cuántos bytes dentro de ese segmento específico quieres acceder.

Segmentación paginada

La segmentación paginada es un híbrido que combina lo mejor de paginación y de segmentación.

La paginación tiene la ventaja de que es completamente transparente para el programador además de eliminar la fragmentación externa. Mientras que la segmentación en cambio es visible para el programador quien pensará el código, datos, heap, stack, etc pero la desventaja principal causará fragmentación externa.

La *segmentación paginada* resuelve el dilema de ambos métodos, cada segmento es dividido internamente en páginas de tamaño fijo. Así el programador verá los segmentos granularmente pero internamente cada uno está dividido en páginas eliminado la fragmentación externa y permitiendo que un segmento sea más flexible en su distribución física.

Con paginación vimos que el espacio de direcciones de un proceso no necesariamente debe estar contiguo para poder ejecutarse si no que el HW traduce direcciones lógicas a físicas usando las tablas de páginas que el SO administra.

Motivación para memoria virtual

Podríamos pensar que no todo el espacio de direcciones del proceso en todo momento por ejemplo rutinas o librerías que se ejecutan una única vez o partes del programa que no vuelven a ejecutarse por lo que significa que no hay necesidad que la totalidad de imagen del proceso sea cargada en memoria.

El SO puede traer a memoria las “piezas” de un proceso a medida que este las necesita lo cual se define como **conjunto residente** a la porción del espacio de direcciones del proceso que se encuentra en memoria.

Con el apoyo del HW se detecta cuando se necesita una porción del proceso que no está en su conjunto residente por lo cual se debe cargar en memoria para continuar con la ejecución.

Lo positivo de esto es que

- Más procesos pueden ser mantenidos en memoria ya que solo son cargadas algunas secciones de cada proceso y con más de estos en memoria principal será más probable que existan más procesos ready.
- Un proceso puede ser más grande que la memoria principal pero el usuario no se debe preocupar por el tamaño de sus programas.

Para memoria virtual el hardware debe soportar

- Paginación por demanda/ segmentación por demanda.
- Tener un dispositivo de memoria secundaria que de el apoyo para almacenar las secciones del proceso que no están en memoria principal.
- El SO debe ser capaz de manejar el movimiento de las páginas entre la memoria principal y la secundaria.

Memoria virtual con paginación

La memoria virtual con paginación es el sistema de gestión de memoria más usado en los SO modernos ya que combina las ventajas de la paginación con la capacidad de usar el disco como extensión de la RAM.

El concepto fundamental es que cada proceso tiene su propia tabla de páginas. Esta tabla es como un mapa personal de cada proceso que especifica dónde está cada una de sus páginas. El tamaño de la tabla de páginas depende del tamaño del espacio de direcciones del proceso.

Cada entrada en la tabla referencia al frame o marco en el que se encuentra la página en la memoria principal. Cada entrada en la tabla de páginas tiene bits de control.

- Bit V: indica si la página está en memoria.
- Bit M: indica si la página fue modificada. Si se modificó, en algún momento, se deben reflejar los cambios en memoria secundaria.

Fallo de páginas

Ocurre cuando el proceso intenta usar una dirección que está en una página que no se encuentra en la memoria principal, `Bit V= 0`. El HW lo detecta y genera un trap en el SO, el cual podrá colocar al proceso en estado blocked mientras gestiona que la página que se necesite se cargue.

El SO buscará un frame o marco libre en la memoria y genera una operación de E/S al disco para copiar en dicho frame la página del proceso que se necesita utilizar. El SO puede asignarle la CPU a otro mientras otro proceso mientras se completa la E/S.

Cuando la operación de E/S finaliza, se notifica al So y este actualiza la tabla de páginas del proceso indicando que la página está en memoria.

El proceso que generó el fallo de página vuelve a ready y cuando se ejecute, volverá a ejecutar la instrucción que antes generó el fallo de página.

Tabla de páginas

Cada proceso tiene su tabla de páginas cuyo tamaño de cada una depende del espacio de direcciones del proceso, como fue mencionado anteriormente y que puede alcanzar un tamaño considerable.

Podrán organizarse de diferentes maneras según las necesidades del sistema.

- Tabla de 1 nivel: tabla única y lineal que tiene una entrada por cada página posible del espacio de direcciones. Es simple pero problemático en sistemas de 64 bits.
- Tabla de 2 niveles: divide la tabla en una estructura jerárquica. En lugar de una gigante, hay una tabla de primer nivel que apunta a múltiples tablas de segundo nivel. Cuando buscás una página, primero indexás en la tabla de primer nivel para obtener la tabla de segundo nivel correspondiente y luego se indexa para obtener el marco.
- Tabla invertida (hashing): en lugar de tener una entrada por página lógica, tenés una entrada por marco físico. Usa una función hash para buscar rápidamente. Esto es especialmente útil en sistemas de 64 bits.

Tamaño de la página

- Pequeño: menor fragmentación interna, más páginas requeridas por proceso, generando tablas de páginas más grandes y más páginas pueden residir en memoria.
- Grande: mayor fragmentación interna, la memoria secundaria está diseñada para transferir grandes bloques de datos más eficientemente.

Translation lookaside buffer

El problema fundamental es que cada acceso a memoria con paginación requiere múltiples accesos a memoria física. Cuando un proceso accede a una dirección lógica, el HW primero necesita acceder a la tabla de páginas para obtener el número de marco y luego acceder a RAM nuevamente. Con tablas multinivel es peor porque primero accede a tabla de primer nivel, luego a la tabla de segundo nivel y finalmente a los datos, lo que significa que una operación simple como una lectura podría causar varios accesos a RAM.

Considerando que la RAM es relativamente lenta (comparada con la CPU), esto causaría un cuello de botella severo. Sin solución, el sistema sería muy lento porque la CPU estaría constantemente esperando a que RAM le devuelva información sobre dónde están las páginas.

La solución es un cache de HW de alta velocidad conocido como TLB (translation lookaside buffer). TLB es una pequeña memoria que almacena traducciones recientemente usadas de direcciones lógicas a marcos físicos.

Dada una dirección virtual, el procesador examina la TLB y si la entrada de la tabla de páginas se encuentra en la TLB es obtenido el frame y armada la dirección física.

Si la entrada no está en la TLB, el número de página es usado como índice en la tabla de páginas del proceso.

Se controla si la página está en la memoria y si no lo está, generará un page fault. La TLB es actualizada para incluir la nueva entrada y cuando haya un cambio de contexto habrá una invalidación de las entradas de la TLB.

Asignación de marcos

Un concepto importante en paginación es cuántos marcos de RAM puede ocupar cada proceso simultáneamente. Esto se llama el **tamaño del conjunto residente** (resident set size): cuántas páginas de un proceso pueden estar en memoria principal en un momento dado.
Existen dos enfoques principales para decidir cuántos marcos asignar a cada proceso.

- Asignación dinámica: el número de marcos asignados a cada proceso varía. Inicialmente, cuando un proceso se crea, se le asigna un número mínimo de marcos. A medida que el proceso ejecuta y necesita más páginas, el SO le asigna marcos adicionales. Si el proceso necesita menos memoria, algunos marcos pueden ser recuperados y reasignados a otros procesos. Esto es flexible: si un proceso necesita mucha memoria en un momento dado, obtiene más marcos. Si luego necesita menos, esos marcos se liberan para otros procesos. La ventaja es mejor utilización del sistema: la memoria se distribuye dinámicamente según las necesidades actuales. La desventaja es que requiere más gestión del SO.
- Asignación fija: cada proceso recibe un número fijo de marcos cuando se crea y mantiene ese número durante toda su ejecución. Por ejemplo, todo proceso obtiene exactamente 100 marcos, sin importar cuánta memoria realmente necesite. Es simple de implementar: no hay que ajustar marcos continuamente. Pero es inflexible: un proceso pequeño que solo necesita 50 marcos desperdicia 50 marcos, mientras que un proceso grande que necesita 150 marcos solo tiene 100 y sufrirá muchos page faults.
  Los sistemas modernos típicamente usan asignación dinámica porque es más eficiente, aunque con mecanismos para evitar que un proceso acapare toda la RAM.

Reemplazo de páginas

Cuando ocurre un page fault y todos los marcos están ocupados, el SO debe liberar un marco para traer la página faltante. Esto significa elegir una página existente y desalojarla al disco. La pregunta es: **¿cuál página desalojar?**

El reemplazo óptimo sería desalojar la página que nunca más será referenciada o no pronto. El problema es cómo predecir esto, por lo cual se usarán algoritmos que a partir del comportamiento del pasado podrán saber cuál es la página más adecuada para desalojar.

Los algoritmos más comunes son

- LRU (least recently used) que desaloja la página que no ha sido accedida durante más timepo.
- FIFO (first in first out) que desaloja la página más antigua.
- Clock, simula un reloj para aproximarse a LRU de forma más eficiente.

Alcance del reemplazo

El reemplazo de páginas puede ocurrir de diferentes formas según el alcance (scope) que el SO considere.

- Reemplazo global: el SO no distingue entre procesos cuando elige qué página desalojar. Cuando un proceso sufre un page fault, el SO busca la página menos recientemente usada en toda la RAM. Las consecuencias son

significativas. Un proceso que tiene un page fault puede terminar desalojando páginas de otros procesos, no solo de sí mismo. Esto significa que el rendimiento de un proceso no depende solo de su propio comportamiento, sino también del comportamiento de otros procesos ejecutándose simultáneamente.

Un proceso de **alta prioridad podría tomar marcos de un proceso de baja prioridad**, favoreciendo al proceso importante. Por ejemplo, un proceso de sistema crítico podría causar page faults que desalojen páginas de un proceso de usuario, asegurando que el proceso crítico tenga suficientes marcos para ejecutar eficientemente.

La ventaja del reemplazo global es que es eficiente: maximiza la utilización de memoria total. La desventaja es que es impredecible: no tienes control sobre cuántos marcos tu proceso tendrá en cada momento.

- Reemplazo local: el fallo de página de un proceso solo puede reemplazar sus propias páginas lo cual no cambia la cantidad de frames asignados y el SO puede determinar cuál es la tasa de page-faults de cada proceso. Un proceso puede tener frames asignados que no usa y no pueden ser usados por otros procesos.

Algoritmos de reemplazo

- Óptimo: es solo teórico.
- FIFO: el más sencillo.
- LRU (least recently used): requiere soporte del hardware para mantener timestramps de acceso a las páginas. Favorece a las páginas menos recientemente accedidas.
- 2da chance: un avance del FIFO tradicional que beneficia a las páginas más referenciadas.
- NRU (non recently used): utiliza bits R y M que favorece a las páginas que fueren usadas recientemente.

Thrashing (hiperpaginación)

Decimos que un sistema está en thrashing cuando pasa más tiempo paginando que ejecutando procesos y como consecuencia hay una baja importante en el sistema.

1. El kernel monitorea el uso de la CPU.
2. Si hay baja utilización, aumenta el grado de multiprogramación.
3. Si el algoritmo de reemplazo es global, pueden sacarse frames a otros procesos.
4. Un proceso necesita más frames y comienzan los page-faults y robo de frames a otros procesos.
5. Por swapping de páginas y encolamiento en dispositivos, baja el uso de la CPU.
6. Vuelve a 1.

Control de thrashing

Se puede limitar el thrashing usando algoritmos de reemplazo local. Con este algoritmo si un proceso entra en thrashing no roba frames a otros procesos, si bien perjudica la performance, es controlable.

Si un proceso cuenta con todos los frames que necesita, no había trashing. Caso de que ocurra, para abarcar la problemática se puede usar la estrategia de **working set** la cual se apoya en el modelo de localidad o la otra estrategia será **PFF (frecuencia de fallos en página).**

- Modelo de localidad: la idea centran es que las referencias a datos y código tiendan a agruparse en el tiempo y espacio. La localidad de un proceso en un momento dado se da por el conjunto de páginas que tiene en memoria en ese momento ya que son las piezas que el proceso necesita para ejecutarse. En cortos períodos de tiempo, un proceso típicamente solo necesita unas pocas páginas para funcionar mientras que el resto permanecen en disco.

  Un programa se compone de varias localidades por lo que para prevenir la hiperactividad es necesario que un proceso tenga en memoria sus páginas más activas para evitar page faults.

- Modelo working set: basado en el modelo de localidad para gestionar eficientemente la memoria virtual. Tendrá el parámetro de v**entana del working set Δ** que es un parámetro que define un período de tiempo reciente. Será como una ventana que mira hacia atrás en el tiempo para ver qué páginas fueron accedidas recientemente.
    - Δ chico no cubrirá la localidad.
    - Δ grande puede tomar varias localidades.

Prevención del thrashing con working set

La **prevención del thrashing** usa el modelo de working set para ser inteligente. El SO monitorea continuamente cada proceso, calculando cuál es su working set actual (WSS). Básicamente, el SO cuenta cuántas páginas únicas el proceso necesita para funcionar eficientemente basándose en sus accesos recientes.
El SO entonces **asigna a cada proceso suficientes marcos para contener su working set completo**. Si el proceso pi necesita un WSS de 10 páginas, le asigna al menos 10 marcos. Esto garantiza que cuando el proceso ejecute, tendrá sus páginas necesarias en RAM sin page faults constantes.

Si quedan marcos libres después de asignar los working sets, el SO puede iniciar otro proceso pero si la suma de todos los working sets excede la memoria disponible, el SO elige un proceso para suspender completamente, reasignando sus marcos a otros procesos. Este proceso suspendido no ejecuta en absoluto hasta que haya más memoria disponible, pero al menos los otros procesos pueden ejecutar eficientemente sin thrashing.

Prevención de thrashing por PFF

La técnica PFF (page fault frecuency) en lugar de calcular el WS, usa la tasa de fallos de página para estimar si el proceso tiene un conjunto residente que representa adecuadamente al WS.

Es ideal establecer límites interiores y superiores de los PFF deseados, si se excede el PFF máximo entonces se le asigna un frame más al proceso porque este exceso indica que hay muchos page faults y probablemente lo esté necesitando en cambio si la PFF está por debajo del mínimo, se le sacará un frame porque tiene de más.

Puede pasar el caso que se suspenda un proceso si no hay más frames y estos se reasignarán a otros procesos con alto PFF.

Demonio de paginación

El **demonio de paginación** (page daemon) es un proceso especial creado por el SO durante el arranque que trabaja en segundo plano para mantener la memoria en condiciones óptimas.
Se ejecuta automáticamente cuando el sistema detecta que la memoria está bajo estrés: cuando hay poca memoria libre disponible, o cuando demasiadas páginas han sido modificadas y aún no están guardadas en disco.
Algunas tareas importantes serán

- Limpiar páginas modificadas sincronizándolas con el swap. Cuando un proceso modifica una página, esa página debe ser guardada en disco en algún momento para preservar los cambios. El demonio busca páginas modificadas y las escribe al disco (al área de swap). Esto es importante porque reduce el tiempo que tardará después si esa página necesita ser desalojada: si ya está guardada en disco, se desaloja instantáneamente. Si no estuviera guardada, el SO tendría que escribirla antes de desalojarla, lo cual es lento.
- Reducir el tiempo de transferencia al sincronizar varias páginas contiguas. En vez de escribir una página a la vez, el demonio agrupa páginas contiguas y las escribe juntas en una sola operación.
- Mantener un número mínimo de marcos libres en el sistema. El demonio monitorea cuántos marcos están disponibles. Si está por debajo del umbral, el demonio desaloja páginas para liberar marcos evitando que haya una salutación.
- Demora la liberación de una página hasta que haga falta. El demonio mantiene las páginas en memoria durante un tiempo, si se necesita pronto, está en RAM pero si esta está bajo presión, desaloja las que estuvieron libres por mucho tiempo.

Memoria compartida

Gracias al uso de la tabla de páginas, varios procesos pueden compartir un marco de memoria el cual debe estar asociado a una página en la tabla de páginas de cada proceso. Esto permite compartir código entre procesos pero los datos privados a cada uno no podrán ubicarse en esta memoria compartida.

Mapeo de archivo en memoria

El **mapeo de archivo en memoria** permite que un proceso acceda un archivo como si fuera memoria. En lugar de leer el archivo con system calls, el proceso simplemente accede direcciones de su memoria y el SO automáticamente trae los datos del archivo.

El archivo no se carga completo a memoria. Solo se cargan las partes que el proceso accede. Cuando el proceso intenta acceder a una parte no cargada, ocurre un page fault y el SO trae esa parte del archivo a memoria.

La ventaja es que es muy eficiente: parece que trabajas con memoria pero en realidad estás accediendo un archivo, sin que el proceso deba hacer nada especial.

Cuando el proceso termina o libera el archivo mapeado, el SO escribe automáticamente todas las páginas modificadas en el archivo. No tienes que hacer nada manualmente; los cambios se guardan solos.
El mapeo de archivo en memoria es una forma alternativa de hacer entrada/salida sin usar las operaciones tradicionales del sistema de archivos. En lugar de leer y escribir explícitamente, trabajas con memoria y el SO se encarga de sincronizar con el archivo.
Se usa comúnmente para compartir librerías o DLLs entre múltiples procesos. Si varios procesos necesitan la misma librería, pueden mapearla en memoria. El SO carga solo una copia en RAM y la comparte entre todos, ahorrando memoria.

Copia en escritura

La copia en escritura permite a los proceso padre e hijo compartir inicialmente las mismas páginas en memoria y si un hijo modifica una página compartida, la pagina será copiada. Podrá crearse procesos de forma más eficiente debido a que solo las páginas modificadas son duplicadas.

Área de intercambio

El área de intercambio o swap es el espacio en disco que el SO usa como extensión de la RAM. Cuando la memoria física se llena, el SO mueve páginas al área de swap para liberar RAM.

Hay dos técnicas para administrar esta área

1. La primera **reserva espacio de una vez**: cuando se crea un proceso, el SO reserva en disco una zona del tamaño de la imagen completa del proceso. Cada página virtual del proceso tiene una dirección de disco asignada. Cuando una página se intercambia, se escribe en esa ubicación. Esto es simple pero desperdicia espacio en disco porque reservas para todas las páginas aunque nunca las uses todas.
2. **No se asigna nada inicialmente:** solo cuando una página realmente necesita ir al disco, el SO le asigna espacio en ese momento. Esto ahorra espacio en disco, pero requiere llevar registro de dónde está cada página en disco, lo cual consume más memoria para las tablas de control. El SO debe mantener un registro que mapee cada página virtual a su ubicación en disco.

Clase 4

Un SO debe controlar dispositivos de E/S mediante generación de comandos, manejo de interrupciones y errores además de proporcionar una interfaz de utilización.

Al tener dispositivos muy diversos y complejos de gestionar se presentarán algunos problemas

- Heterogeneidad de dispositivos: hay distinto tipos y funcionamientos.

- Variación en las características de los dispositivos.
- Velocidad, son más lentos que la CPU.
- Hay nuevos tipos de dispositivos constantemente.
- Hay diferentes formas de realizar E/S.

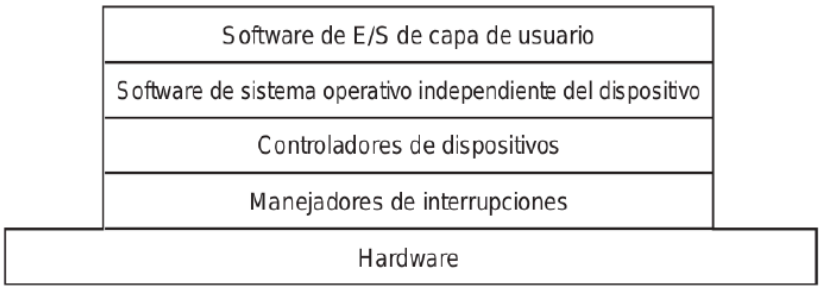
Aspectos que caracterizan los dispositivos de E/S

- Data-transfer mode: si se transfieren carácter por carácter o en bloques completos.
- Access method: si se acceden a los datos secuencialmente o aleatoriamente.
- Transfer schedule: si es síncrona o asíncrona.
- Sharing: si el dispositivo es dedicado (solo un proceso lo usa a la vez, como una impresora) o compartible (múltiples procesos lo usan simultáneamente, como un teclado).
- Device speed: latencia (tiempo de espera antes de empezar), seek time (tiempo para llegar al dato), transfer rate (velocidad de lectura) y delay (esperas entre operaciones).
- I/O direction: si el dispositivo es de lectura, escritura o ambas.

Es deseable manejar todos los dispositivos de E/S uniforme y estandarizadamente. Es ideal ocultar la mayoría de los detalles del dispositivo en las rutinas de niveles más bajos para que los procesos vean a los dispositivos en término de operaciones comúnes como read, write, open, close, etc.

Los dispositivos de E/S pueden resultar lentos respecto a la memoria y la CPU por lo cual la multi-programación permite que un proceso espere por la finalización de su E/S mientras que otro proceso se ejecuta por lo que será necesario la planificación de estos dispositivos para minimizar tiempos.

- **Buffering** será el almacenamiento de los datos en memoria mientras se transfieren con el objetivo de resolver problemas de velocidad, tamaño y forma de los datos.
- **Caching**: mantiene en memoria copia de los datos de reciente acceso para mejorar la performance.
- **Spooling**: administra al cola de requerimientos de un dispositivo. Algunos de acceso exclusivo no pueden atender distintos requerimientos al mismo tiempo lo cual es un mecanismo para coordinar el acceso concurrente al dispositivo.
- Manejo de errores: el SO debe administrar los errores ocurridos mediante números de errores o logs.
- Formas de realizar E/S
    - Bloqueante: el proceso se suspende hasta que el requerimiento de E/S se completa lo cual será fácil de usar y entender pero no suficiente en algunos casos.
    - No bloqueante: el requerimiento de E/S retorna cuando es posible.



El diagrama es la arquitectura en capas de la E/S desde el HW hasta el usuario.

1. Hardware: dispositivos físicos reales como discos, impresoras, teclados, etc.
2. Manejadores de interrupciones: cuando un dispositivo completa una operación o necesita atención, genera una interrupción. Estos manejadores interceptan esa interrupción y dicen al SO qué dispositivo la generó y qué pasó.
3. Controladores de dispositivos: programas que saben cómo comunicarse con cada dispositivo.
4. SW del SO independiente del dispositivo: proporciona una interfaz uniforme para el SO sin importar el dispositivo que hay debajo.
5. Software de E/S de capa de usuario: aplicaciones y bibliotecas que el usuario ejecuta.

Performance

E/S es uno de los factores que más afectan la performance el sistema ya que

- Usa mucho la CPU para ejecutar los drivers y el código del subsistema de E/S.
- Provoca cambios de contexto ante interrupciones y bloqueos de procesos.
- Usa el bus de memoria en copia de datos.

Mejorar la performance sería posible mediante

- Reducción del número de context switches.
- Reducción de la cantidad de copias de datos mientras se pasan del dispositivo a la aplicación.
- Reducción de frecuencia de interrupciones con transferencia de gran cantidad de datos, tener controladores más inteligentes y usando polling.

Clase 5

Usamos archivos para almacenar grandes cantidades de datos, almacenar a largo plazo y permitir a distintos procesos acceder al mismo conjunto de información.

Archivo

Es una entidad abstracta con nombre el cual ocupa un espacio lógico continuo y direccionable.

Puntos de vista del usuario

- Operaciones que se pueden llevar a cabo.
- Cómo nombrar a un archivo.
- Cómo asegurar la protección.
- Cómo compartir archivos.
- No trata con aspectos físicos.

Punto de vista del diseño

- Implementar archivos.
- Implementar directorios.
- Manejar el espacio en disco.
- Manejar el espacio libre.
- Observar la eficiencia y mantenimiento.

Sistema de manejo de archivos

Es un conjunto de unidades e software que proveen los servicios necesarios para la utilización de archivos como crear, borrar, buscar, copiar, leer, escribir, etc.

Además, facilita el acceso a los archivos por parte de las aplicaciones y la abstracción para el programador.

Objetivos del SO en cuanto a archivos

- Cumplir con la gestión de datos.
- Cumplir con las solicitudes del usuario.
- Minimizar/ eliminar la posibilidad de perder o destruir datos.
- Dar soporte de E/S a distintos dispositivos.
- Brindar un conjunto de interfaces de E/S para tratamiento de archivos.

Tipos de archivos

- Archivos regulares como texto plano o binarios.
- Directorios: archivos que mantienen la estructura en el FileSystem.

Atributos de un archivo

1. Nombre.
2. Identificador.
3. Tipo.
4. Localización.
5. Tamaño.
6. Protección, seguridad y monitoreo: owner, permisos, password, registrar modificaciones, etc.

Directorios

Contienen información acerca de archivos y directorios que están dentro de él siendo en sí mismo un archivo.

Interviene en la resolución entre el nombre y el archivo mismo mediante las operaciones de buscar, crear, borrar, listar, renombrar archivos, etc.

El uso de directorios ayudará con la eficiencia mediante la localización rápida de archivos, el uso del mismo nombre y la agrupación lógica de archivos por propiedades o funciones.

Estructura de directorios

Los archivos pueden ubicarse siguiendo un path desde el directorio raíz y sus sucesivas referencias (full pathname) o path absoluto.

Distintos archivos pueden tener el mismo nombre pero el fullpathname es único.

El directorio actual en el cual nos ubicamos se lo llama “directorio de trabajo” y dentro del directorio de trabajo, se pueden referenciar los archivos tanto por su PATH absoluto como por su PATH relativo indicando solamente la ruta al archivo desde el directorio de trabajo.

Identificación absoluta y relativa

Los archivos y directorios se pueden identificar de dos maneras diferentes.

- **Identificación absoluta**: especifica el camino completo desde la raíz al sistema de archivos hasta el archivo. La ruta absoluta siempre especifica dónde está el archivo sin importar dónde estés actualmente.

- **Identificación relativa**: especifica el camino desde el directorio en el que estás actualmente.

Compartir archivos

En un ambiente multiusuario se necesita que varios usuarios puedan compartir archivos por lo que debe ser realizado bajo un esquema de protección con derechos de acceso y manejo de accesos simultáneos.

Respecto a la protección, el propietario debe ser capaz de controlar qué se puede hacer y quién lo puede hacer.

Los directorios también tienen permisos, los cuales pueden permitir el acceso al mismo para que el usuario pueda usar el archivo siempre y cuando tenga permisos.

- Execution: el usuario puede ejecutar.
- Reading: el usuario puede leer el archivo.
- Appending: el usuario puede agregar datos pero no modificar o borrar el contenido del archivo.
- Updating: el usuario puede modificar, borrar y agregar archivos.
- Changing protection: el usuario puede modificar los derechos de acceso.
- Deletion: el usuario puede borrar el archivo.
- Owners.

Metas del sistema de archivos

- Brindar espacio en disco a los archivos de usuario y del sistema.
- Mantener un registro del espacio libre.

Conceptos

- Sector: unidad de almacenamiento usada en discos rígidos.
- Bloque/clusters: conjuntos de sectores consecutivos.
- FileSystem: define la forma en que los datos almacenados.
- FAT (file allocation table): contienen información sobre en qué lugar están alocados los distintos archivos.

Pre-asignación

Se necesita saber cuánto espacio va a ocupar el archivo en el momento de su creación y por lo general se tiende a definir espacios mucho más grandes que lo necesario esto significa la posibilidad de utilizar sectores contiguos para almacenar los datos de un archivo. Será necesario estudiar el caso en el que el archivo supera el espacio asignado.

Surge la **asignación dinámica** en el cual el espacio se solicita a medida que se necesita pero los bloques de datos pueden quedar de manera no contigua.

Asignación continua

Método para almacenar archivos en disco usando bloques consecutivos. El archivo ocupa un conjunto de bloques del disco. Requiere pre-asignación lo que significa que se debe conocer el tamaño del archivo previamente.

La **File Allocation Table (FAT)** es una tabla que el sistema de archivos mantiene en disco para registrar dónde se está almacenando cada uno. Es muy simple en este método. Solo necesita una entrada por archivo que especifique dos cosas: el bloque de inicio (dónde comienza el archivo en el disco) y la longitud (bloques que ocupa).

La asignación continua además permite que el archivo pueda ser leído con una única operación y su desventaja es que puede generar fragmentación externa la cual se puede gestionar con la compactación.

Asignación encadenada

Es un método alternativo que permite que los archivos crezcan dinámicamente sin necesidad de bloques contiguos.

La asignación se basa en bloques individuales donde cada uno tiene un puntero al siguiente bloque del archivo.

La FAT continúa siendo simple ya que una entrada por archivo contiene solo el bloque de inicio y tamaño del archivo.

No habrá fragmentación externa ya que los bloques están dispersos en cualquier parte del disco y si se necesita más espacio, simplemente se le asigna otro bloque libre en cualquier ubicación y se encadena. Tampoco hay fragmentación interna porque usa exactamente los bloques que necesita sin desperdiciar espacio.

Es útil para el acceso secuencial porque si lee el archivo de principio a fin solamente hay que seguir los punteros.

Asignación indexada

Método que combina velocidad de acceso con flexibilidad. Resuelve el problema de la asignación encadenada donde a partes lejanas del archivo requiere seguir muchos punteros.

En lugar de que cada bloque de datos contenga un puntero al siguiente, existe un bloque índice para cada archivo. La FAT contiene un puntero al bloque índice, no a los datos. El bloque índice no contiene datos del archivo, sino un índice o lista de todos los bloques que componen el archivo.

La ventaja principal es que permite el acceso aleatorio rápido, los bloques no necesitan ser contiguos ni encadenados, crecen bajo demanda y no hay fragmentación externa.

Gestión de espacio libre

Es un control sobre cu[ales de los bloques de disco están disponibles, algunas alternativas son

- **Tablas de bits**: es una tabla con un bit por cada bloque de disco donde Bit= 0, bloque libre, bit= 1, bloque en uso. La ventaja es que es fácil de encontrar un bloque o grupo de bloques libres pero la desventajas es el tamaño del vector en memoria.
- **Bloques libres encadenados**: se tiene un puntero al primer bloque libre y cada uno de ellos tiene un puntero al bloque siguiente libre, similar a lista invertida encadenada de FOD. Resultará ineficiente para la busqueda de bloques libres ya que es necesario varias operaciones de L/E además de que surgirán problemas por pérdida de enlaces además de que es difícil encontrar bloques libres consecutivos.
- Indexación: es una variante del anterior donde el primer bloque libre contiene las direcciones de N bloques libres. Las N-1 direcciones son bloques libres y la N-ésima dirección referencia otro bloque con N direcciones de bloques libres.

UNIX- Manejo de archivos

- Tipos de archivos
  - Archivo común.
  - Directorio.
  - Archivos especializados.
  - Named pipes (comunicación entre procesos).
  - Links (comparten el i-nodo, solo dentro del mismo filesystem).
  - Links simbólicos (tiene i-nodo propio, para filesystems diferentes).

Cada disco físico puede ser dividido en uno o más volúmenes. Cada volumen o partición contiene un sistema de archivos. Cada sistema de archivos contiene

- Boot block: código para iniciar el S.O.
- Superblock: atributos sobre el FileSystem (bloques/clusters libres).
- I-NODE table: tabla que contiene todos los i-nodos.

    - I-NODO: estructura de control que contiene la información clave de un archivo.
- Data blocks: bloques de datos de los archivos.

Clase 6

Disk cache

Buffers en memoria principal para almacenamiento temporario de bloques de disco con el objetivo de minimizar la frecuencia de acceso al disco.

Cuando un proceso quiere acceder a un bloque de la cache hay dos alternativas

1. Se copia el bloque al espacio de direcciones del usuario $\longrightarrow$ no permitiría compartir el bloque.
2. Se trabaja como memoria compartida permitiendo acceso a varios procesos. Esta área de memoria debe ser limitada, con lo cual debe existir un algoritmo de reemplazo.

Estrategia de reemplazo

Cuando se necesita un buffer para cargar un nuevo bloque, se elige el que hace más tiempo que no es referenciado.

Es una lista de bloques, donde el último es el más recientemente usado y cuando un bloque se referencia o entra en la cache, queda la final de la lista.

No se mueven bloques en memoria si no que se asocian punteros.

Buffer cache - Unix system V

El objetivo es minimizar la frecuencia de acceso a disco y es una estructura formada por buffers donde el kernel asigna un espacio en la memoria durante la inicialización para esta estructura.

El buffer tiene dos partes

- Header: contiene información del bloque, número, estado, relación con otros buffers, etc.
- Buffer en sí: el lugar donde se almacena el bloque de disco traído en memoria. Tiene varios estados entre los cuales son free o busy.

Free list: organiza los buffers disponibles para ser utilizados para cargar nuevos bloques de disco. No necesariamente los buffers están vacíos. Sigue el mismo esquema de la hash queue pero confiene los headers de los buffers de aquellos procesos que ya han terminado. El header de un buffer siempre está en la hash queue. Si el procesos que lo referenciaba terminó, va a estar en la hash queue y en la free list.

Hash queues: son colas para optimizar la búsqueda de un buffer en particular. Los headers de los buffers se organizan según una función de hash usando. Al número de bloque se le aplica una funció'n de hash que permite agrupar los buffers cuyo resultado dio igual para hacer que las búsquedas sean más eficientes.

Funcionamiento del buffer cache

Cuando un proceso quiere acceder a un archivo, utiliza su i-nodo para localizar los bloques de datos donde se encuentra éste. El requerimiento llega al buffer cache quien evalúa si puede satisfacer el requerimiento o si debe realizar la E/S.

Se pueden dar 5 escenarios

1. El kernel encuentra el bloque en la hash queue y el buffer está libre.
2. El kernel no encuentra el bloque en la ash queue y utiliza un buffer libre.
3. Ídem 2, pero el bloque libre está marcado como DQ.

4. El kernel no encuentra el bloque en la hash queue y la free list está vaía.
5. El kernel encuentra el bloque en la hash queue pero está BUSY.